# volatile and the Java Memory Model

10 min read · Updated 28 Jul 2026 · Topic 38 of 65

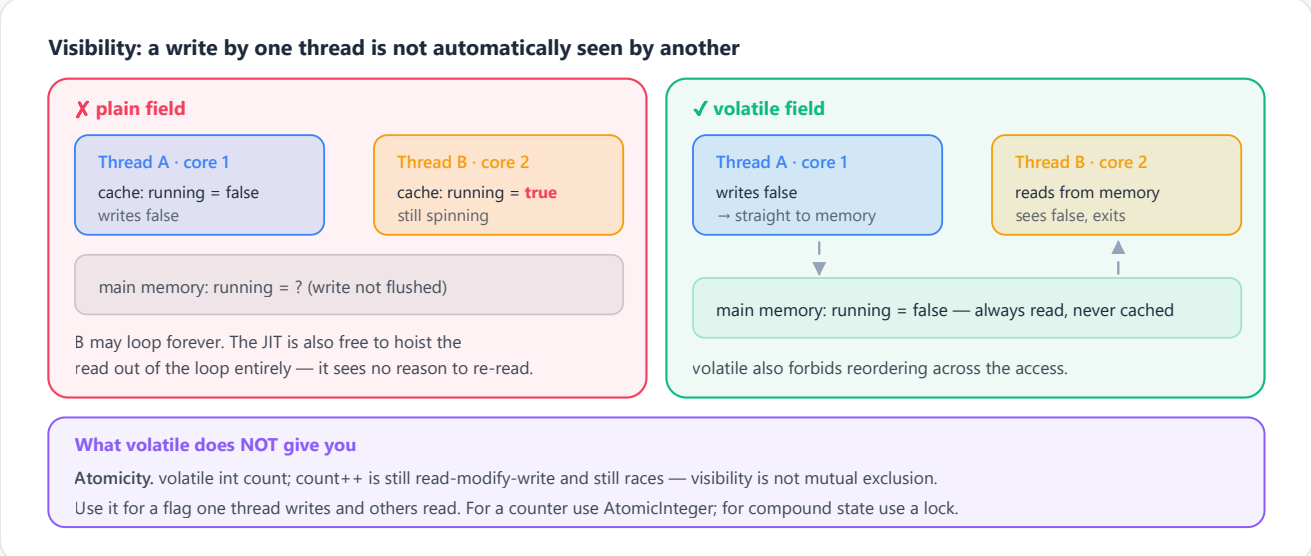
Bookmark · Mark done

## Why it matters

Visibility bugs are the worst class of concurrency bug because they are invisible: no exception, no corrupted output, just a thread that never notices a flag changed and loops forever. They also cannot be reproduced reliably — the same code works in a debugger and hangs in production, because the JIT optimises differently once the method gets hot. The Java Memory Model is what makes reasoning about this possible, and `volatile` is its simplest tool.

## The concept, simply

Each CPU core has its own cache. A write by one thread may sit in that core's cache indefinitely, so another thread reading the same field can see a stale value — potentially forever. On top of that, both the compiler and the CPU may **reorder** instructions, as long as single-threaded behaviour is unchanged.



`volatile` declares that a field must always be read from and written to main memory, and that reads and writes may not be reordered across it. It gives you **visibility and ordering — but not atomicity**.

## How it works

The JMM is defined in terms of **happens-before**: if action A happens-before action B, then everything A did is visible to B. The relationships worth memorising:

| Rule | Meaning |
|---|---|
| Program order | Within one thread, statements happen-before later ones |
| Monitor lock | Unlocking happens-before any later lock of the same monitor |
| `volatile` | A write happens-before every later read of that field |
| Thread start | `start()` happens-before anything in the new thread |
| Thread join | Everything in a thread happens-before `join()` returns |
| `final` fields | Correct construction makes them visible without synchronisation |
| Transitivity | A → B and B → C implies A → C |

The `volatile` rule is stronger than "flush the cache". A volatile *write* publishes **everything the thread wrote before it**, and a volatile *read* makes all of it visible. That is what makes the double-checked locking idiom work, and why `volatile` on one field can safely publish an entire object graph.

**What `volatile` does not do:** it does not make compound operations atomic. `volatile int count; count++` is still read-modify-write and still races. Use `volatile` for a value one thread writes and others read; use an atomic or a lock when readers also write.

Code examples

The infinite loop:

JAVA

Copy

```
class Worker implements Runnable {
    private boolean running = true;          // ✗ not volatile

    public void run() {
        while (running) { }                  // may NEVER exit
        System.out.println("stopped");
    }
    public void stop() { running = false; }
}
// The JIT sees nothing inside the loop that could modify "running", so it
// entitled to hoist the read out — compiling it to "if (running) while(tr
// The fix is one keyword:
private volatile boolean running = true;     // ✓ always re-read, no hoist
```

Visibility versus atomicity, the distinction that matters most:

JAVA

Copy

```
volatile int count = 0;
count++;            // ✗ STILL A RACE — read, add, write are three steps.
                    //   volatile makes each read and write visible, not t
                    //   sequence atomic.

// ✓ For a counter:
AtomicInteger count = new AtomicInteger();
count.incrementAndGet();

// ✓ volatile IS the right tool here — single writer, many readers, and th
//   new value does not depend on the old one:
private volatile Config config;
public void reload(Config c) { this.config = c; }   // one writer
public Config current() { return config; }          // many readers
```

Double-checked locking, the classic example of why `volatile` is more than a cache flush.

JAVA

Copy

```java
// ✗ Broken without volatile. "instance = new Singleton()" is three steps:
//    allocate memory, run the constructor, assign the reference — and the
//    JIT may REORDER the last two. Another thread can then see a non-null
//    reference to a half-constructed object.
private static Singleton instance;

// ✓ volatile forbids that reordering.
private static volatile Singleton instance;

public static Singleton getInstance() {
    if (instance == null) {                    // fast path, no lock
        synchronized (Singleton.class) {
            if (instance == null) {            // re-check under the lock
                instance = new Singleton();
            }
        }
    }
    return instance;
}

// ✓ Honestly, prefer the holder idiom (Chapter 2) — no volatile, no
//   double-check, and the JVM guarantees the laziness and the thread safe
private static class Holder { static final Singleton INSTANCE = new Single
public static Singleton getInstance() { return Holder.INSTANCE; }
```

Safe publication, which is the practical form of all this:

JAVA

Copy

```java
// An immutable object with final fields can be shared across threads with
// NO synchronisation — the JMM guarantees any thread seeing a properly
// constructed object sees its final fields fully initialised.
public final class Config {
    private final String host;
    private final int port;
    public Config(String host, int port) { this.host = host; this.port = p
```

## Under the hood

`volatile` compiles to **memory barriers** (fences) that constrain both the compiler and the CPU. On x86, whose hardware memory model is already strong, a volatile read is nearly free and a volatile write costs roughly a locked instruction; on weaker architectures such as ARM the barriers are more expensive. So "volatile is cheap" is platform-dependent, though far cheaper than a contended lock either way.

**Why the JMM exists at all:** without a specification, "what can a thread see?" would depend on the CPU architecture and the JIT's mood, and portable concurrent code would be impossible. JSR-133 (Java 5) rewrote the original broken model and gave us the happens-before framework, the `final`-field guarantees, and the volatile semantics that make double-checked locking work. Before Java 5, DCL was broken *even with* `volatile` — a good piece of history to have ready.

**The practical hierarchy** is worth stating as a closing answer: don't share state; if you must share, make it immutable; if it must be mutable and is a single value, use `volatile` or an atomic; only if you need to update several things together should you reach for a lock. Most concurrency bugs come from skipping straight to the last option and then getting the critical section wrong.

## Common mistakes

- **Expecting `volatile` to make `++` atomic.** It gives visibility, not mutual exclusion.
- **Omitting `volatile` on a stop flag.** The loop may never see the change.
- **`volatile` on a mutable object reference.** It publishes the *reference* safely, not the object's contents.
- **Double-checked locking without `volatile`.** Reordering can publish a half-built object.
- **Assuming a debugger reproduces it.** Debugging inhibits the optimisations that cause the bug.

- **Synchronising writes but not reads.** Both sides are needed for a happens-before edge.
- **Letting `this` escape a constructor.** Voids the `final`-field guarantee.

## Interview lens

**Q: "What does `volatile` do?"**

A: Two things. Visibility — reads and writes go to main memory rather than a core's cache, so a write by one thread is immediately visible to others. And ordering — it inserts memory barriers that stop the compiler and CPU reordering operations across the access. What it does not give is atomicity: `volatile int count;` `count++` still races.

**Q: "What is happens-before?"**

A: The JMM's ordering relation. If A happens-before B, everything A did is guaranteed visible to B. It is established by program order within a thread, by unlocking-then-locking the same monitor, by a volatile write then read, by `Thread.start()` and `join()`, and by correct construction for `final` fields — and it is transitive. Without such an edge between two threads, there is no guarantee about what one sees of the other.

**Q: "Why can a loop on a non-volatile boolean never terminate?"**

A: Two reasons that compound. The write may sit in the writing core's cache and never reach the reader. And the JIT, seeing nothing in the loop that could change the field, is entitled to hoist the read out — compiling it into a genuinely infinite loop. Marking the field `volatile` forbids both.

**Q: "When is `volatile` enough, and when do you need a lock?"**

A: `volatile` is enough when a single thread writes and others read, and the new value does not depend on the old — a status flag, a swapped-in config object. You

need an atomic or a lock as soon as the operation is read-modify-write, or when several fields must change together consistently.

**Q: "Why does double-checked locking need `volatile`?"**
A: Because `instance = new Singleton()` is allocate, construct, assign — and the last two can be reordered. Without `volatile`, another thread taking the fast path can see a non-null reference to a half-constructed object. `volatile` forbids that reordering. In practice I would use the initialisation-on-demand holder idiom instead, which gets laziness and thread safety from the JVM's class-initialisation guarantee with no volatile at all.

**Q: "Why are immutable objects thread-safe?"**
A: There is no mutable state to race on, and the memory model's `final`-field rule guarantees that any thread seeing a properly constructed object sees its final fields fully initialised — without synchronisation. The caveat is that "properly constructed" means `this` did not escape during construction; publishing a reference from the constructor forfeits the guarantee.

## Cheat-card

- `volatile` = **visibility + ordering. Not atomicity.**
- A volatile write publishes **everything written before it**; a volatile read makes it all visible.
- happens-before edges: program order · monitor unlock→lock · volatile write→read · `start()` · `join()` · `final` fields · transitive.
- Non-volatile flag → the JIT may **hoist the read** → infinite loop.
- Use `volatile` for **single writer, many readers**, value independent of the old one.
- DCL **requires** `volatile` — better still, use the **holder idiom**.
- Immutable + `final` fields = safe publication, no synchronisation — unless `this` escaped.
- Hierarchy: **don't share → immutable → volatile/atomic → lock**.

1. Does volatile make 'count++' thread-safe?

- [ ] No — volatile gives visibility and ordering, not atomicity; increment is still read-modify-write and still races
- [ ] Yes, but only for int and long
- [ ] Only when combined with final
- [ ] Yes — volatile makes the whole operation atomic

Check answer

2. A loop 'while (running) { }' on a non-volatile boolean may never exit. Why?

- [ ] Because the JVM optimises empty loops away entirely
- [ ] The write may sit in the writing core's cache, and the JIT — seeing nothing in the loop that could change the field — may hoist the read out of the loop entirely
- [ ] Because boolean assignments are not atomic
- [ ] Because the loop body is empty

Check answer

3. Why does double-checked locking require the field to be volatile?

- [ ] It does not — volatile is optional there
- [ ] To make the null check atomic

to a half-constructed object

- ◯ Because static fields are cached per thread

Check answer

‹ Previous
Race Conditions and synchronized

Next ›
Locks: ReentrantLock, ReadWriteLock and Conditions

## LEARN MORE

### ARTICLES & DOCS

- Memory model (JLS §17.4)
  docs.oracle.com
- Atomic access — The Java Tutorials
  docs.oracle.com

### VIDEOS




- Java Happens Before Guarantee - Java Memory Model - Part 2
  Jakob Jenkov




- #4 Java Memory Model | Registers, CPU Cache, Race Condition,Volatile, Synchronized | Part - 1
  Bits of Code by Anand

Hand-picked references for this lesson.